

Project Writeup

1. Chosen Scope and Why For my final project, I decided to focus on the "Hostel & Campus Life" scope. Initially, I thought about doing a general academic assistant, but I realized that hostel rules are something students actually stress about constantly. People are always confused about what triggers a wDAC (Warden Disciplinary Action Committee) or what the exact timings are for opposite-gender visitors. I tested standard ChatGPT on these questions, and it confidently hallucinated generic rules that don't apply to IITB at all. I wanted to build something that would give accurate, grounded answers, or strictly say "I don't know" rather than lie.

2. Data Sources Used My primary data source was the official *IIT Bombay Hostel Rules & Regulations* PDF. Working with real-world PDF data was honestly the most frustrating part of this project. When I first extracted the text, it was full of OCR noise—random dates like "11/07/2025" and fragments of handwritten signatures (like "Abhay") scrawled across the pages.

My initial plan was to also include the Mess Menu, but I quickly realized that extracting tabular data using standard text parsers completely broke the row/column structure, rendering it useless for the vector database. To ensure high-quality retrieval and finish within the week, I dropped the mess menu and focused solely on deeply parsing the rulebook. I used Python's `re` module to strip out the signature fragments and page artifacts before chunking.

3. Chunking Strategy and Why At first, I tried the easiest approach: a fixed-character chunker that split the document every 500 characters. This was a disaster. The rulebook is structured with numbered headings (e.g., "2. Visitors and Gender-Specific Rules") followed by several bullet points. Fixed-character chunking ended up separating the bullet points from their headings. So, if a chunk just contained the text *"Unauthorized overnight stays are strictly prohibited,"* the LLM wouldn't know if that applied to

opposite-gender students, family members, or general guests because the header was left behind in a previous chunk.

To fix this, I implemented a **Semantic (Header-Based) Chunking Strategy**. I wrote a regex pattern `(?=\d\.\s+[A-Z])` to split the document exactly at the major numbered headings. Then, I split each section by its bullet points. The most important thing I did was *metadata injection*—I wrote a loop that appends the parent header string to every single bullet point chunk (e.g., creating a string like `[Category: 2. Visitors] Unauthorized overnight stays...`). Because of this, when I loaded it into FAISS, the vector search was able to map queries directly to the correct context every single time.

4. Known Limitations & What I'd Improve If I had more time to work on this, there are two main things I would improve:

- **Multi-turn Memory:** Right now, the Streamlit app is stateless. If I ask "What time can visitors come?" and then ask a follow-up like "Can they use the washing machine?", the app forgets the context of "visitors" in the second prompt. I'd like to implement a memory buffer to pass recent chat history back into the LLM prompt.
- **Table Parsing:** I still want to get the mess menus working. I learned that I would need to use a library like `pdfplumber` to extract tables cell-by-cell, and then write a script that converts each row into a natural language sentence (e.g., "On Monday, the Breakfast menu includes Poha") before embedding it into the database.